

Cheatsheet de gráficos estáticos

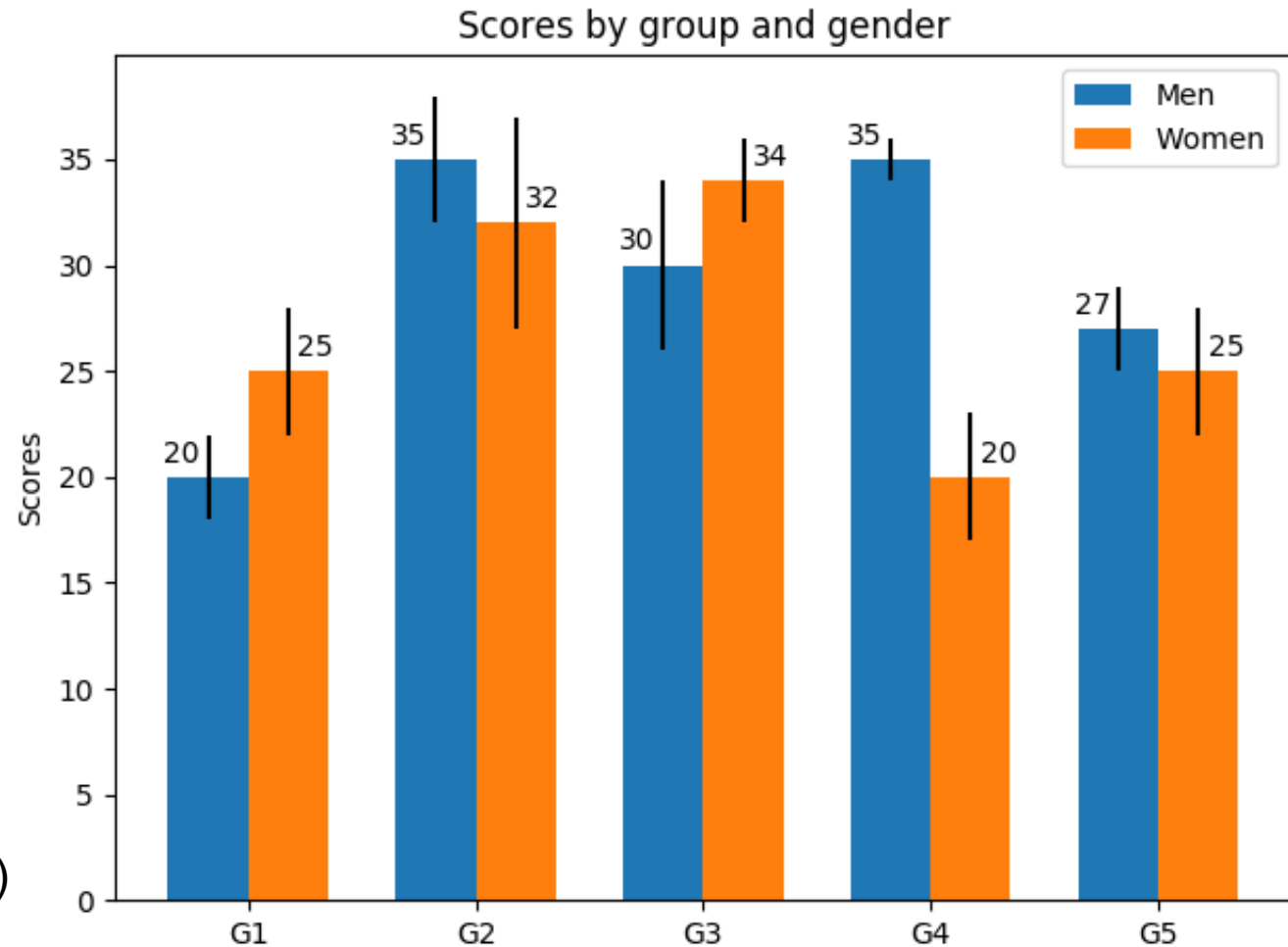
1. Datos categóricos

Comparar magnitudes entre grupos y clasificar frecuencias

Gráfico de barras (Barplot)

El estándar de la industria. Permite comparar de forma directa magnitudes discretas entre grupos de manera muy precisa.

```
import seaborn as sns
sns.barplot(data=df, x='categoria', y='valor')
```



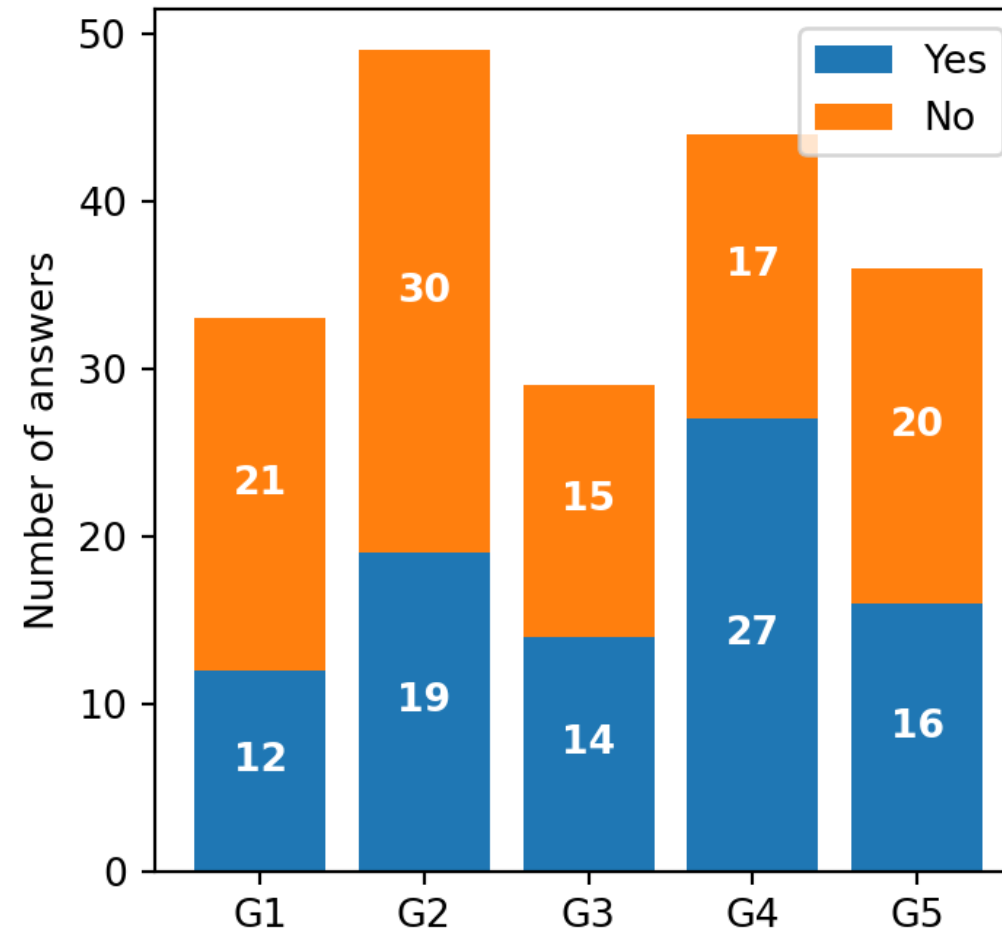
1. Datos categóricos

Comparar magnitudes entre grupos y clasificar frecuencias

Gráfico de barras apiladas (Stacked barplot)

Muestra el volumen total de una categoría y su desglose interno. Útil para ver proporciones, aunque dificulta la comparación visual de los segmentos intermedios.

```
import pandas as pd
df = pd.DataFrame(...)
df.plot(kind='bar', stacked=True)
```



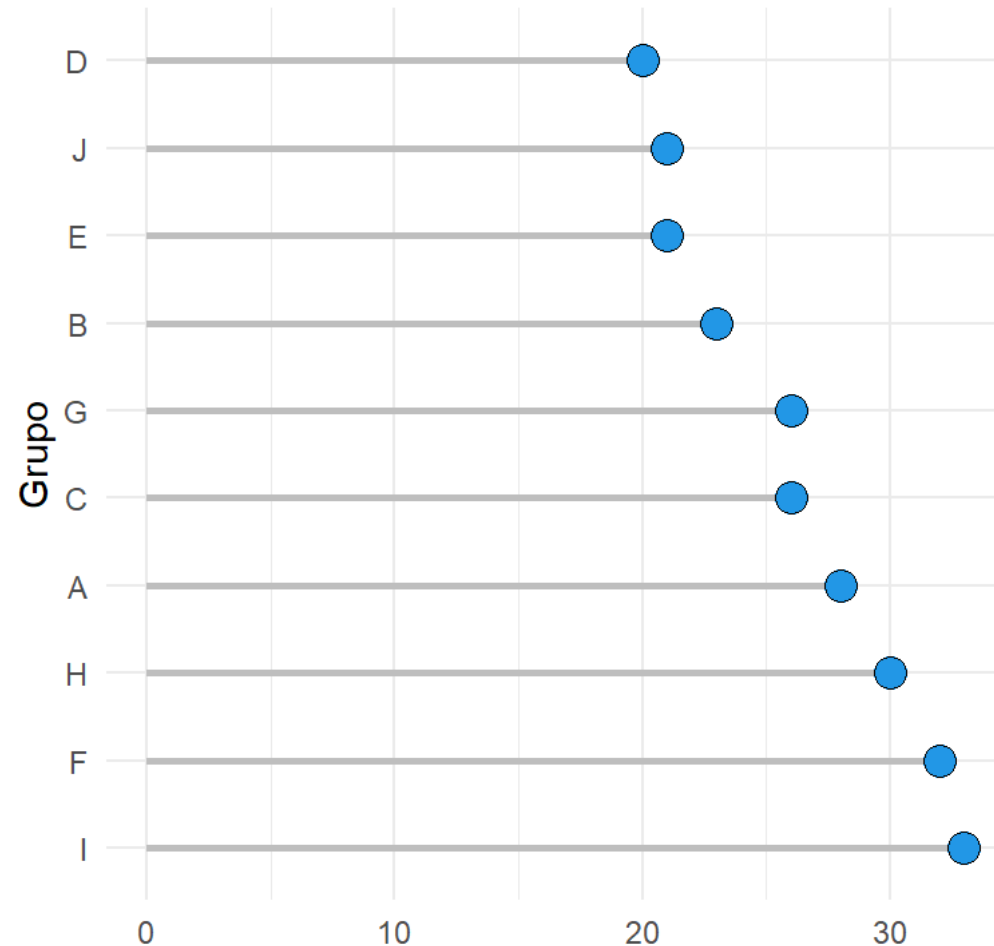
1. Datos categóricos

Comparar magnitudes entre grupos y clasificar frecuencias

Gráfico de piruleta (Lollipop)

Alternativa estilizada. Reduce la tinta total empleada manteniendo el foco en el extremo numérico del valor.

```
import matplotlib.pyplot as plt
plt.stem(df['categoria'], df['valor'])
```



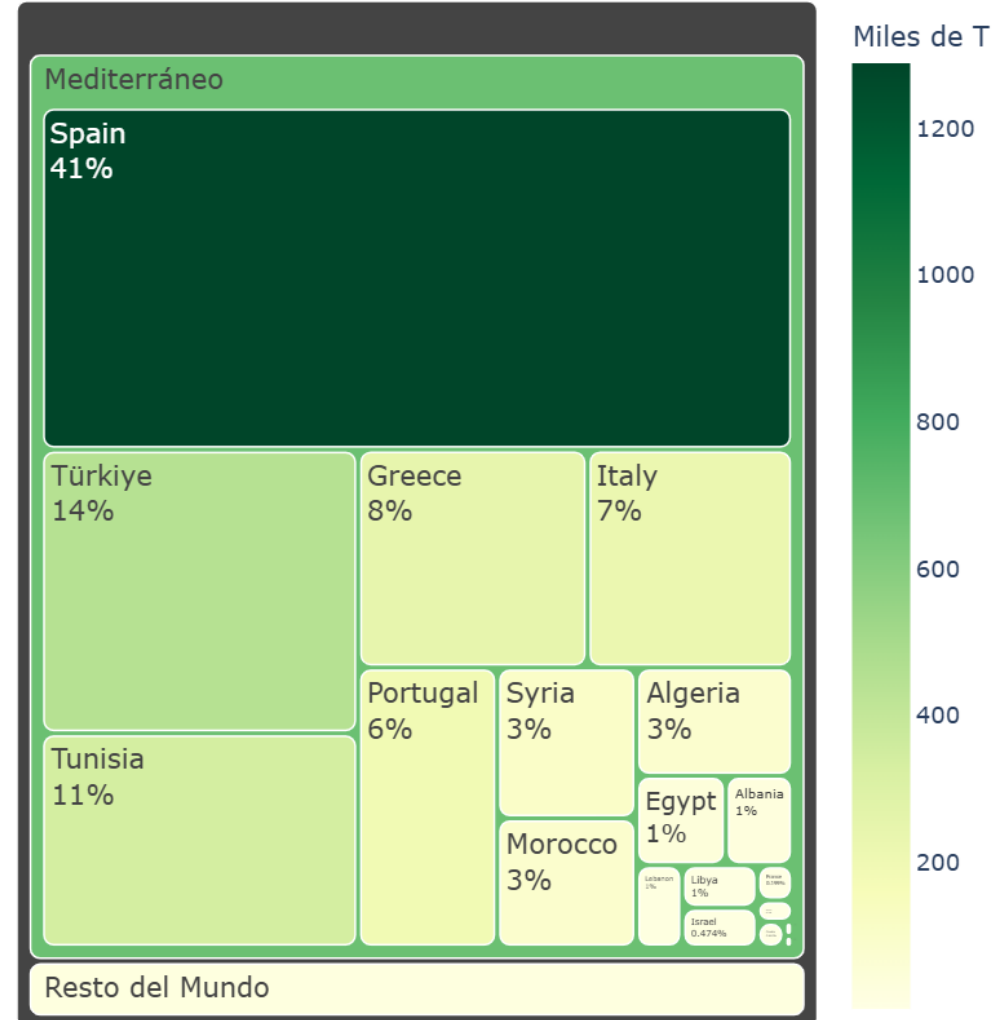
1. Datos categóricos

Comparar magnitudes entre grupos y clasificar frecuencias

Mapa de árbol (Treemap)

Sustituto ideal de sectores con muchas categorías. Codifica el área de forma jerárquica, proporcional, y compacta.

```
import squarify
squarify.plot(sizes=df['valor'], label=df['categoria'])
```



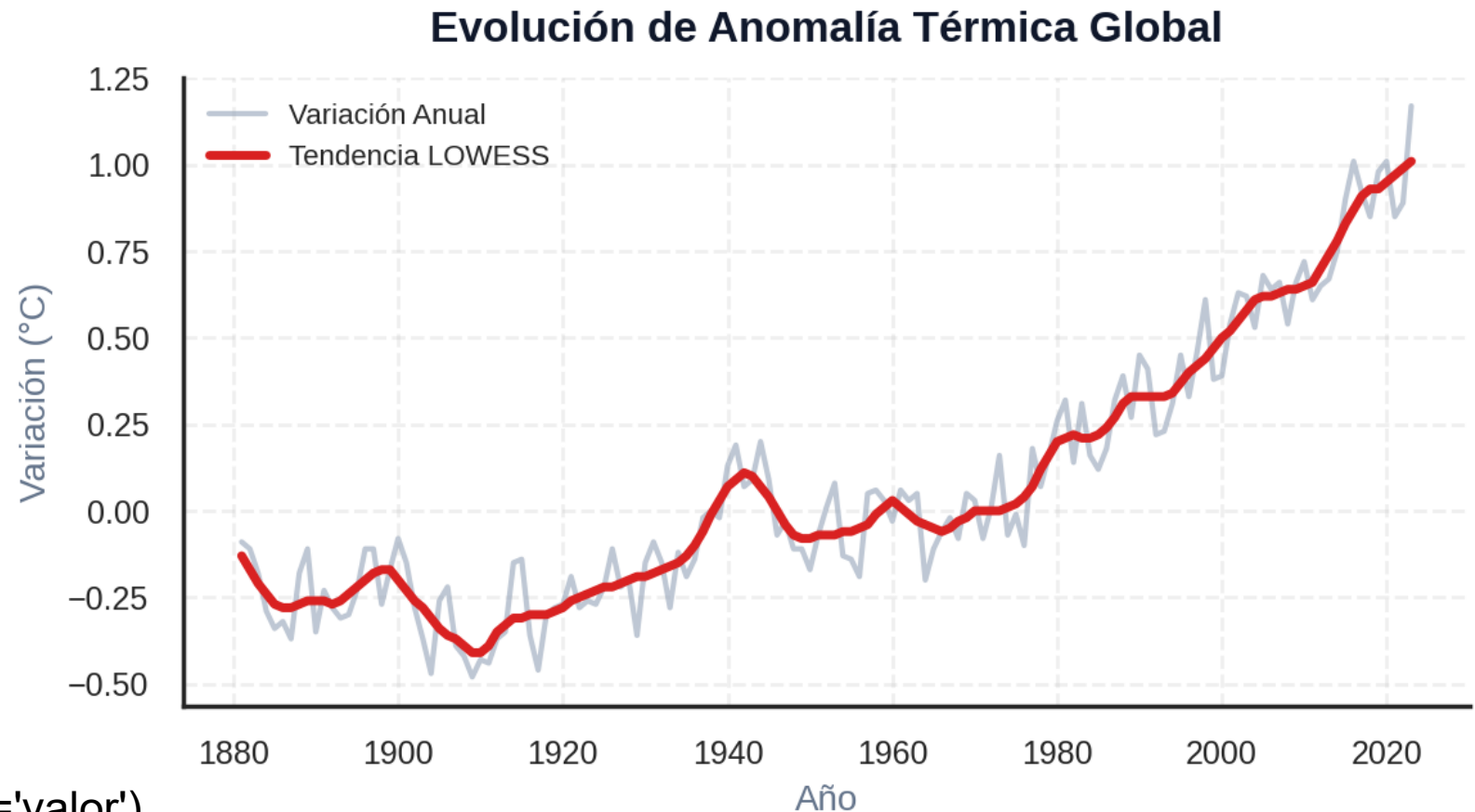
2. Series temporales (Time series)

Mostrar tendencias, ciclos, patrones e intervalos cronológicos

Gráfico de líneas

Ideal para series continuas. Conecta observaciones secuenciales para resaltar tendencias o ciclos cronológicos.

```
import seaborn as sns
sns.lineplot(data=df, x='fecha', y='valor')
```



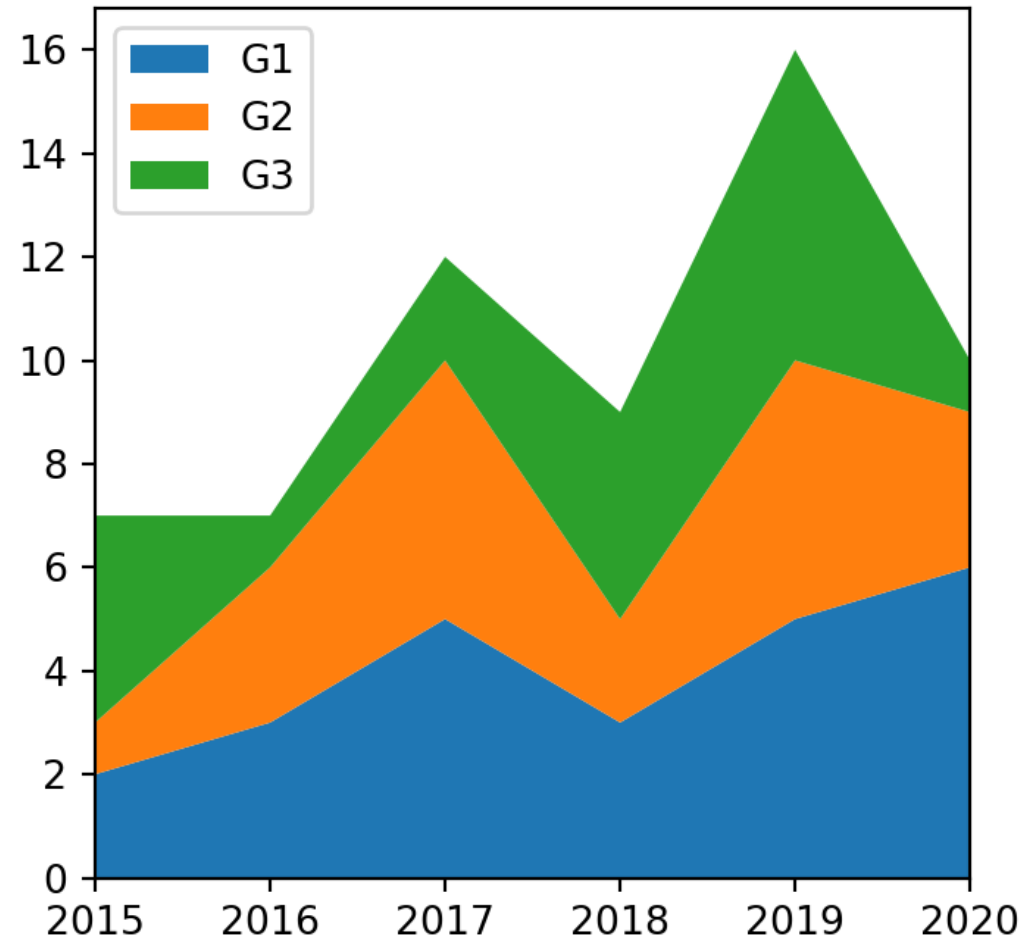
2. Series temporales (Time series)

Mostrar tendencias, ciclos, patrones e intervalos cronológicos

Gráfico de áreas

Rellena el espacio bajo la curva para acentuar el volumen o la masa acumulada global de la variable.

```
import matplotlib.pyplot as plt  
plt.fill_between(df['fecha'], df['valor'])
```



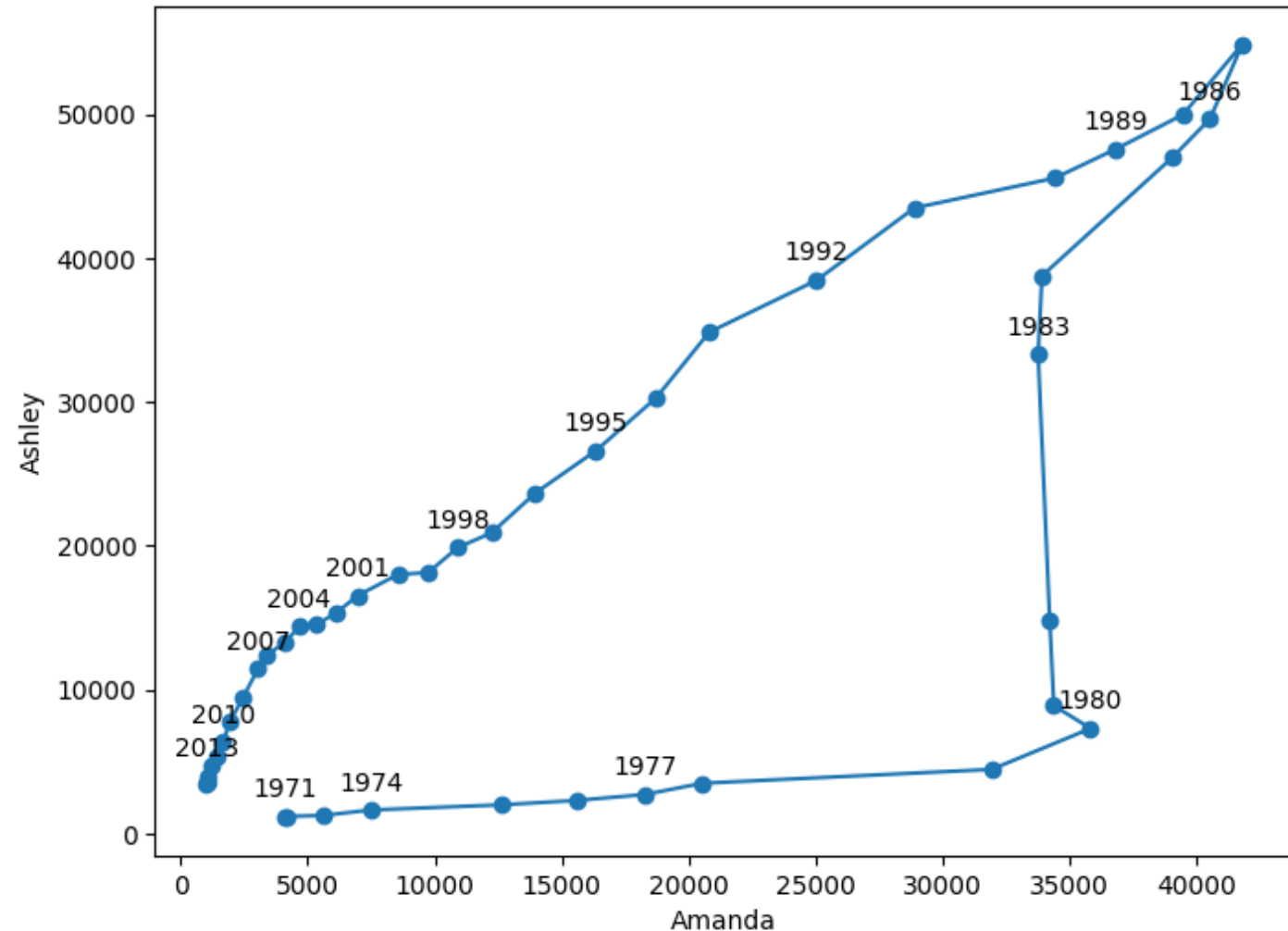
2. Series temporales (Time series)

Mostrar tendencias, ciclos, patrones e intervalos cronológicos

Dispersión conectada (Connected scatterplot)

Útil para representar dos variables (p. ej., nombres) a la vez cuyo vínculo lógico es el orden temporal.

```
import matplotlib.pyplot as plt
plt.plot(df['var_x'], df['var_y'],
marker='o', linestyle='-')
```



3. Datos categóricos y numéricos

Comparar distribuciones estadísticas completas entre categorías

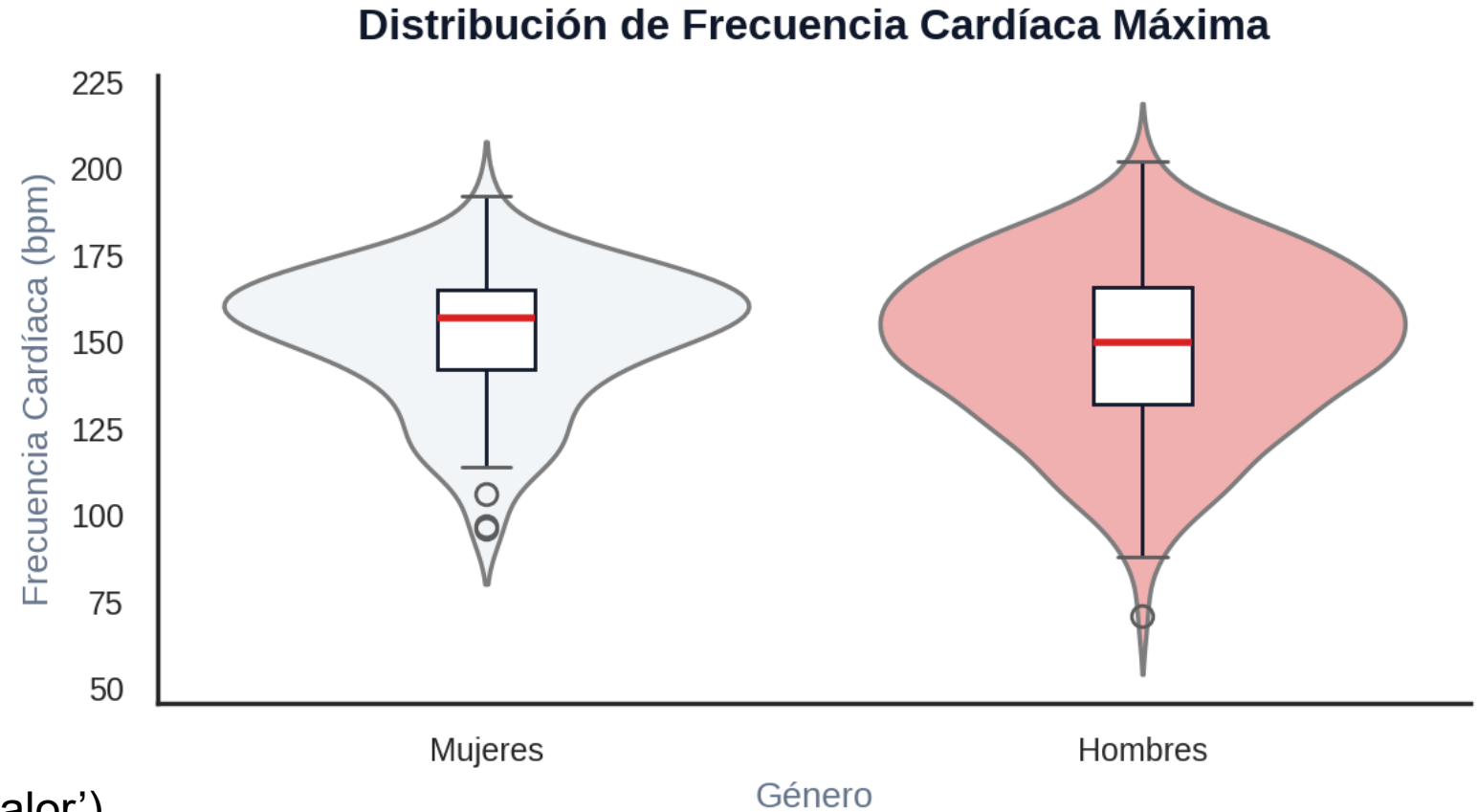
Diagrama de caja y bigotes (Boxplot)

Resumen robusto de 5 puntos (medianas, cuartiles y atípicos). Perfecto para comparar simetrías.

Gráfico de violín

Combina el boxplot con una estimación de densidad kernel (KDE). Revela asimetrías y distribuciones bimodales ocultas.

```
import seaborn as sns
sns.boxplot(data=df, x='categoria', y='valor')
sns.violinplot(data=df, x='categoria', y='valor')
```



3. Datos categóricos y numéricos

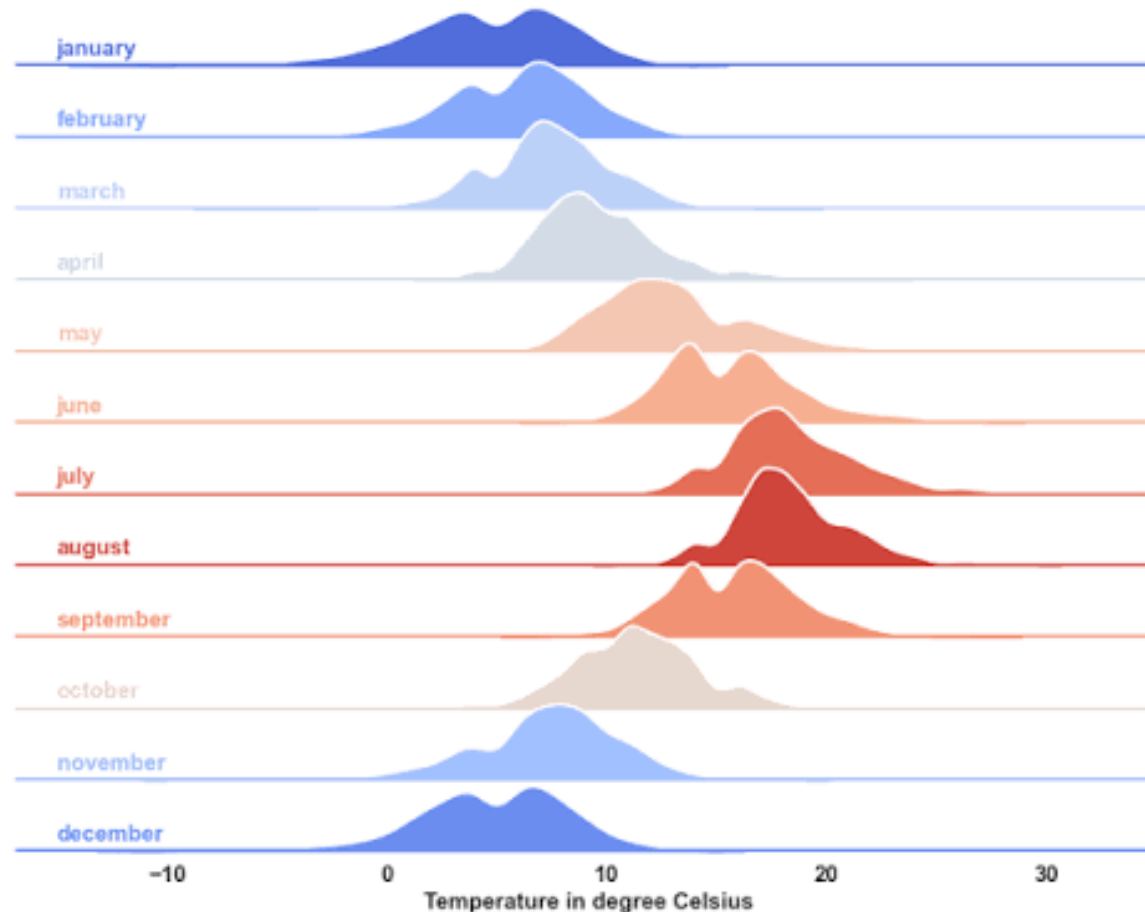
Comparar distribuciones estadísticas completas entre categorías

Gráfico de crestas (Ridgeline)

Solapa de forma vertical múltiples curvas de densidad. Ideal para visualizar cambios graduales o temporales de muchas categorías.

```
import seaborn as sns
sns.FacetGrid(df, row='categoria',
hue='categoria',
aspect=4).map(sns.kdeplot, 'valor')
```

Daily average temperature in Seattle per month



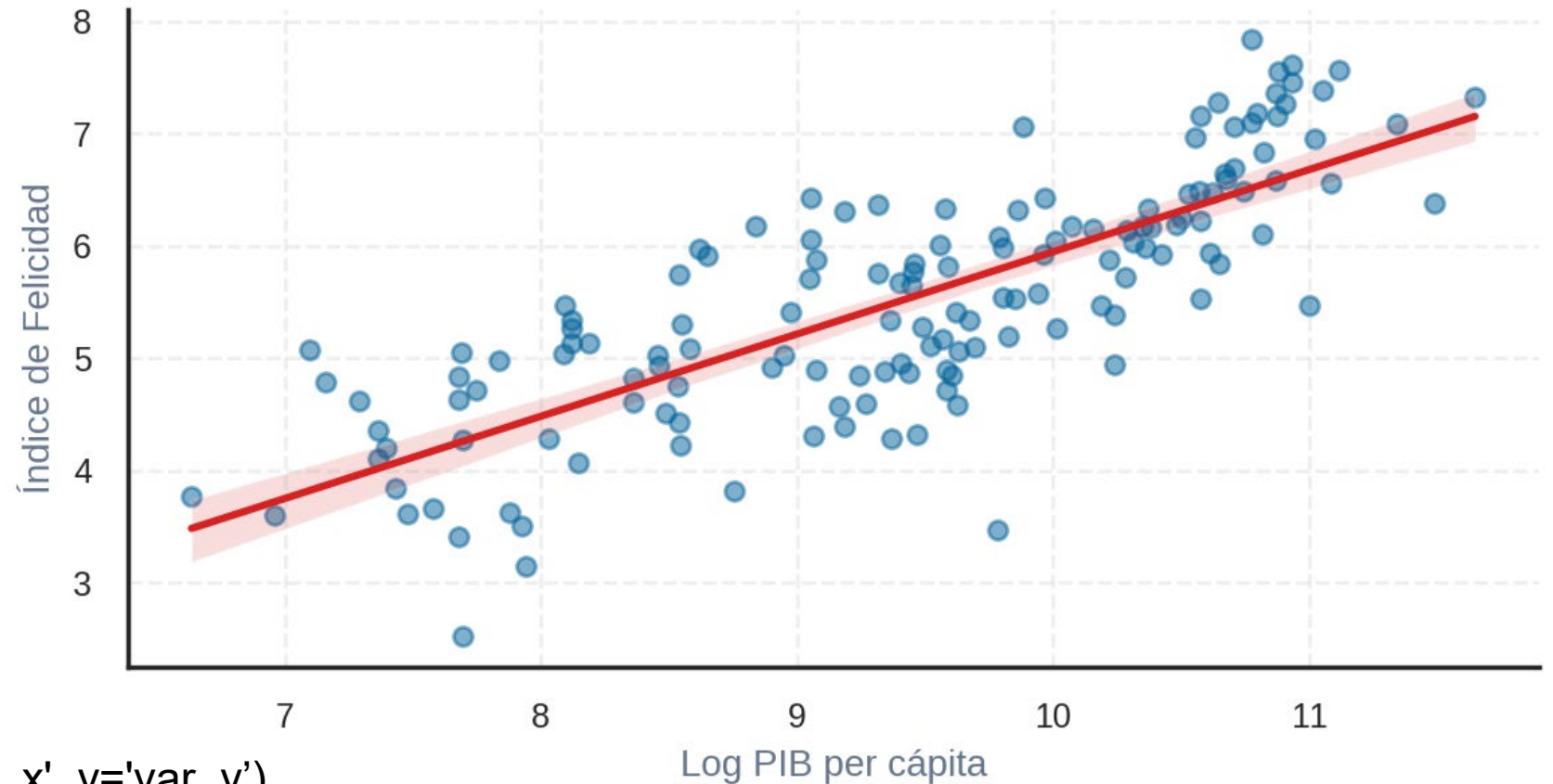
4. Datos numéricos y relaciones

Analizar correlaciones, dispersiones y densidades continuas

Relación entre Riqueza (PIB) y Felicidad

Gráfico de dispersión (Scatterplot)

El rey para buscar correlación bidimensional. La posición en los ejes (X, Y) es el atributo visual que el ojo humano interpreta con mayor exactitud.



```
import seaborn as sns
sns.scatterplot(data=df, x='var_x', y='var_y')
sns.regplot( data=df, x='var_x', y='var_y', ci=95, scatter_kws={'alpha': 0.6}, line_kws={'color': 'red', 'linewidth': 2})
```

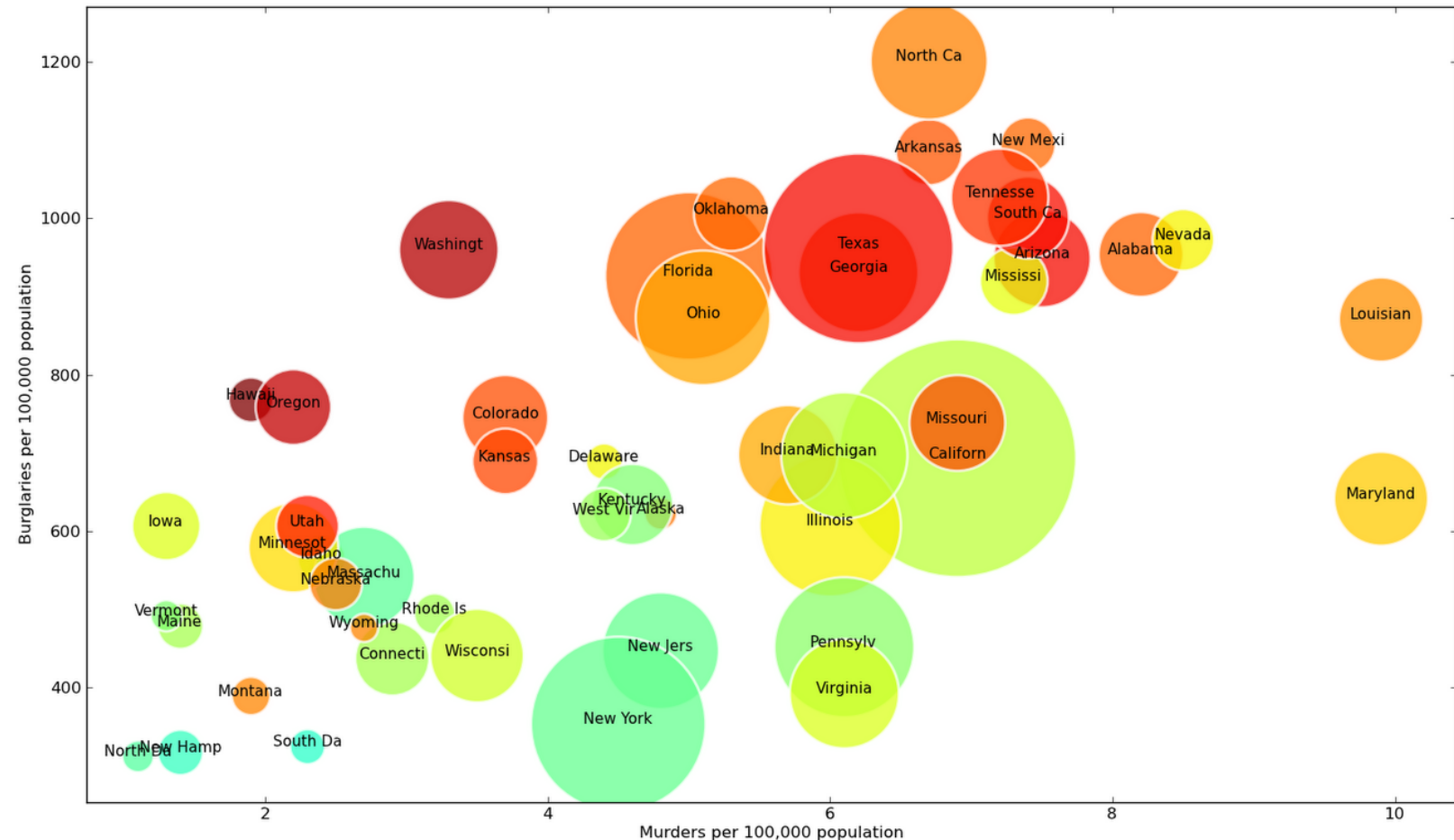
4. Datos numéricos y relaciones

Analizar correlaciones, dispersiones y densidades continuas

Gráfico de burbujas (Bubble)

Integra una tercera dimensión (tamaño del círculo). Cuidado: comparar áreas circulares tiene menor precisión.

```
import seaborn as sns
sns.scatterplot(data=df, x='var_x',
               y='var_y', size='tercera_var',
               sizes=(20, 400))
```



4. Datos numéricos y relaciones

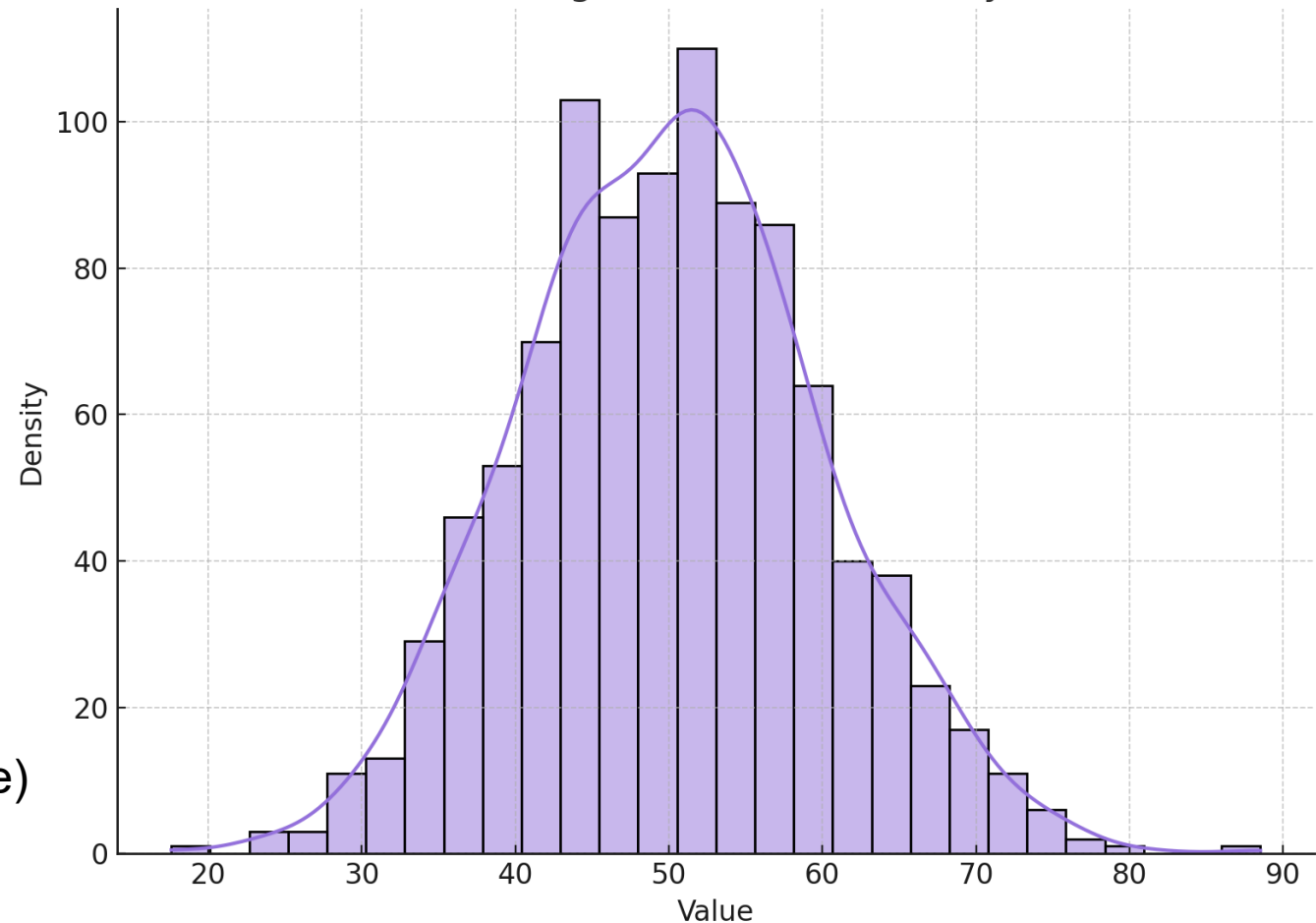
Analizar correlaciones, dispersiones y densidades continuas

Histograma y densidad (KDE)

Segmentan el rango numérico continuo en intervalos ('bins') para cuantificar la forma y la masa de la frecuencia.

```
import seaborn as sns
sns.histplot(data=df, x='valor', kde=True)
```

6. Histogram with KDE Overlay



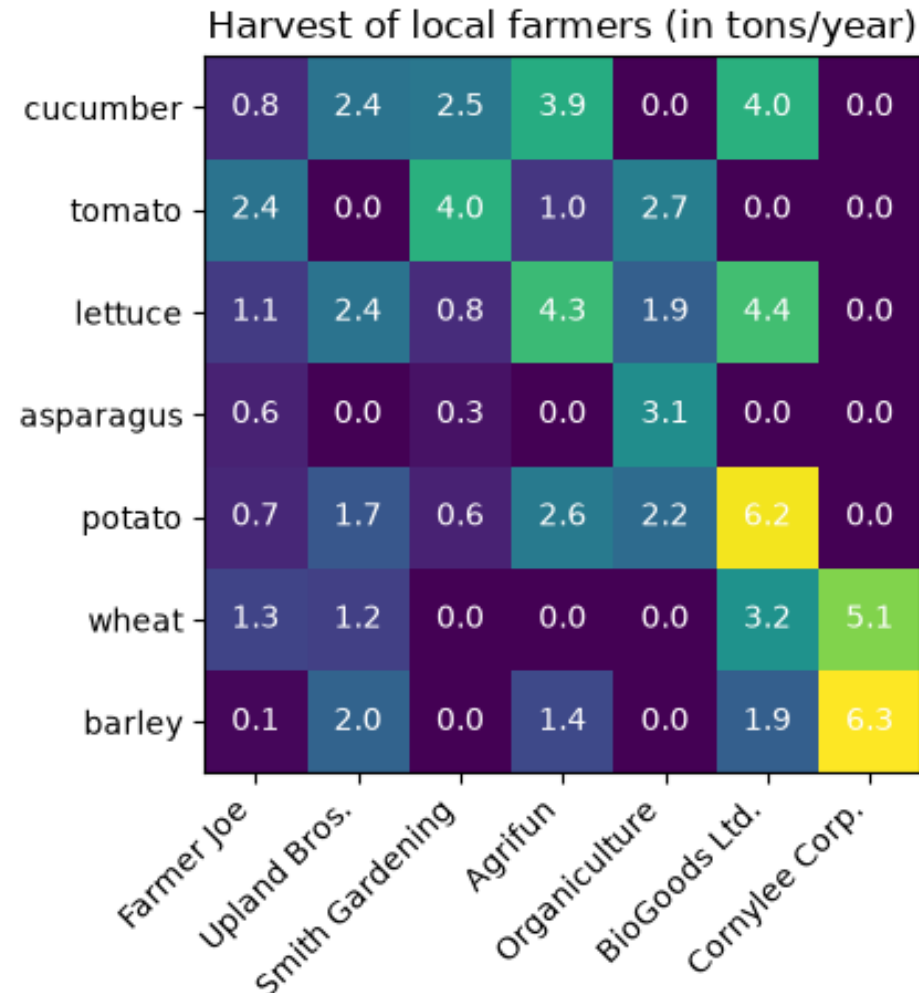
5. Asociación y estructuras multidimensionales

Mapear dependencias recíprocas y colinealidades en matrices

Mapa de calor (Heatmap)

Una cuadrícula rectangular bidimensional donde los valores numéricos se codifican directamente como color.

```
import seaborn as sns
sns.heatmap(data=matriz_datos,
            annot=True, cmap='viridis')
```



5. Asociación y estructuras multidimensionales

Mapear dependencias recíprocas y colinealidades en matrices

Correlograma

Una matriz de correlación visual estructurada para identificar de manera veloz la multicolinealidad antes del modelado de ML.

```
import seaborn as sns
sns.heatmap(df.corr(),
            annot=True, cmap='coolwarm',
            vmin=-1, vmax=1)
```

Matriz de Correlación de Parámetros Clínicos



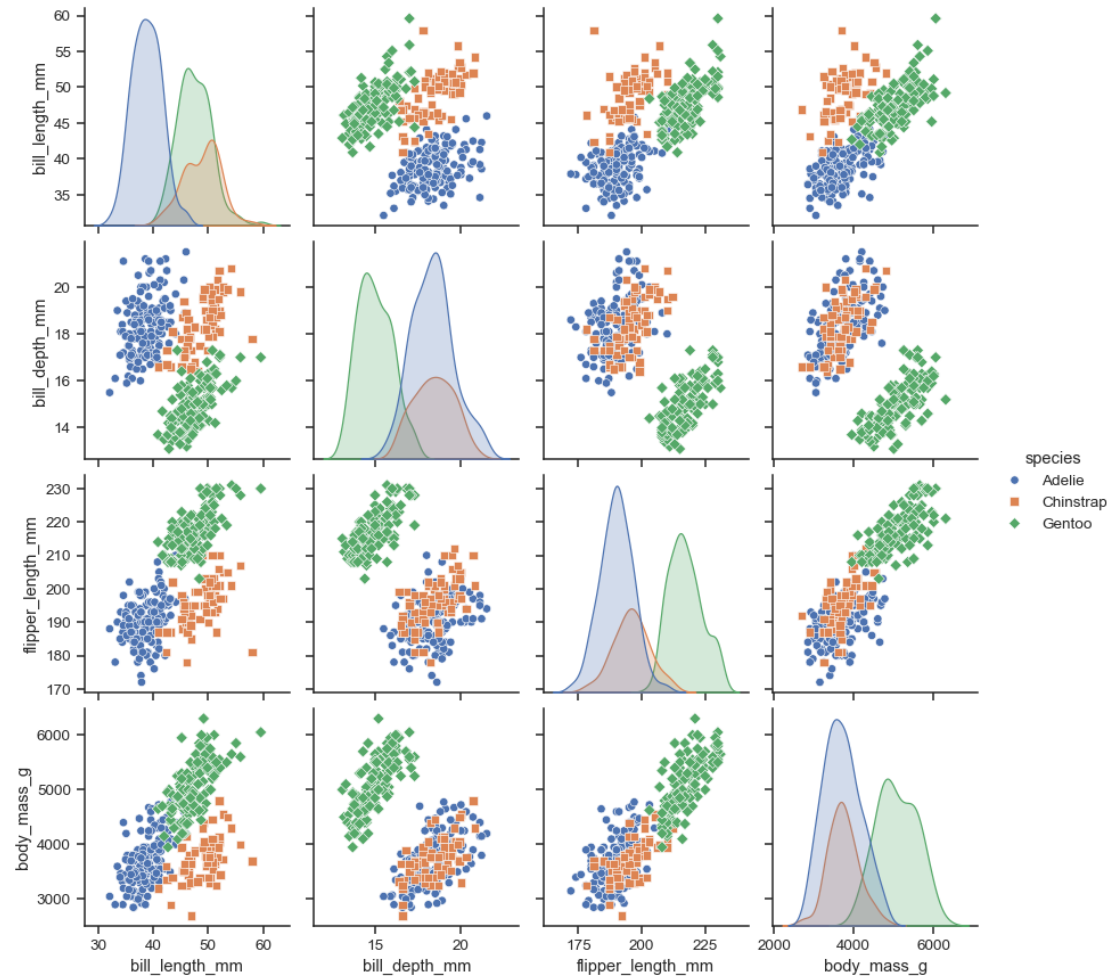
5. Asociación y estructuras multidimensionales

Mapear dependencias recíprocas y colinealidades en matrices

Matriz de dispersión (Pairplot)

Cuadrícula que cruza todas las columnas numéricas de dos en dos: scatterplots en las esquinas para buscar correlaciones y distribuciones (histogramas/KDE) en la diagonal

```
import seaborn as sns
sns.pairplot(data=df)
```



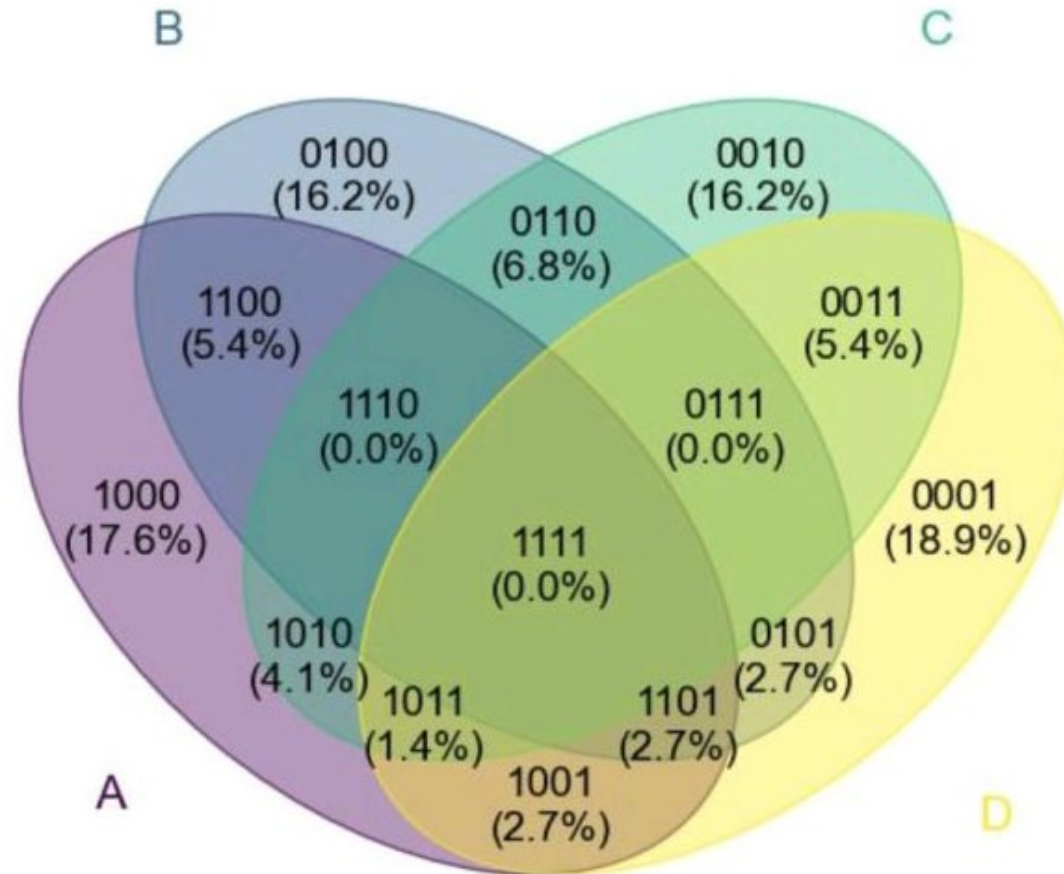
5. Asociación y estructuras multidimensionales

Mapear dependencias recíprocas y colinealidades en matrices

Diagrama de Venn

Visualiza intersecciones lógicas entre conjuntos. Venn es intuitivo para 2-3 grupos; para cruces complejos en Machine Learning.

```
from matplotlib_venn import venn2
venn2(subsets=(10, 5, 2),
set_labels=('Grupo A', 'Grupo B'))
```

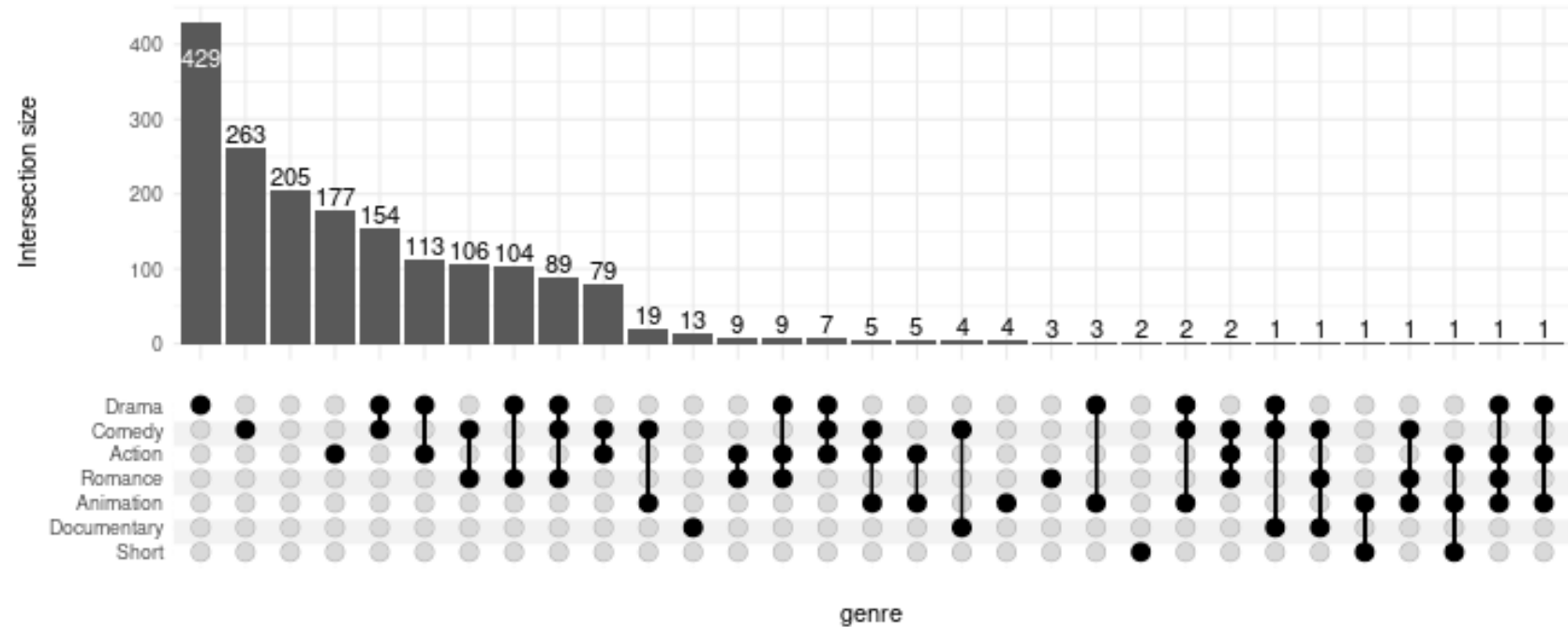
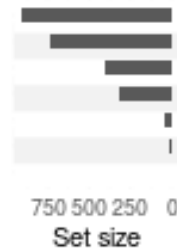


5. Asociación y estructuras multidimensionales

Mapear dependencias recíprocas y colinealidades en matrices

UpSet plot

Visualiza intersecciones lógicas entre conjuntos. UpSet plot es el estándar.



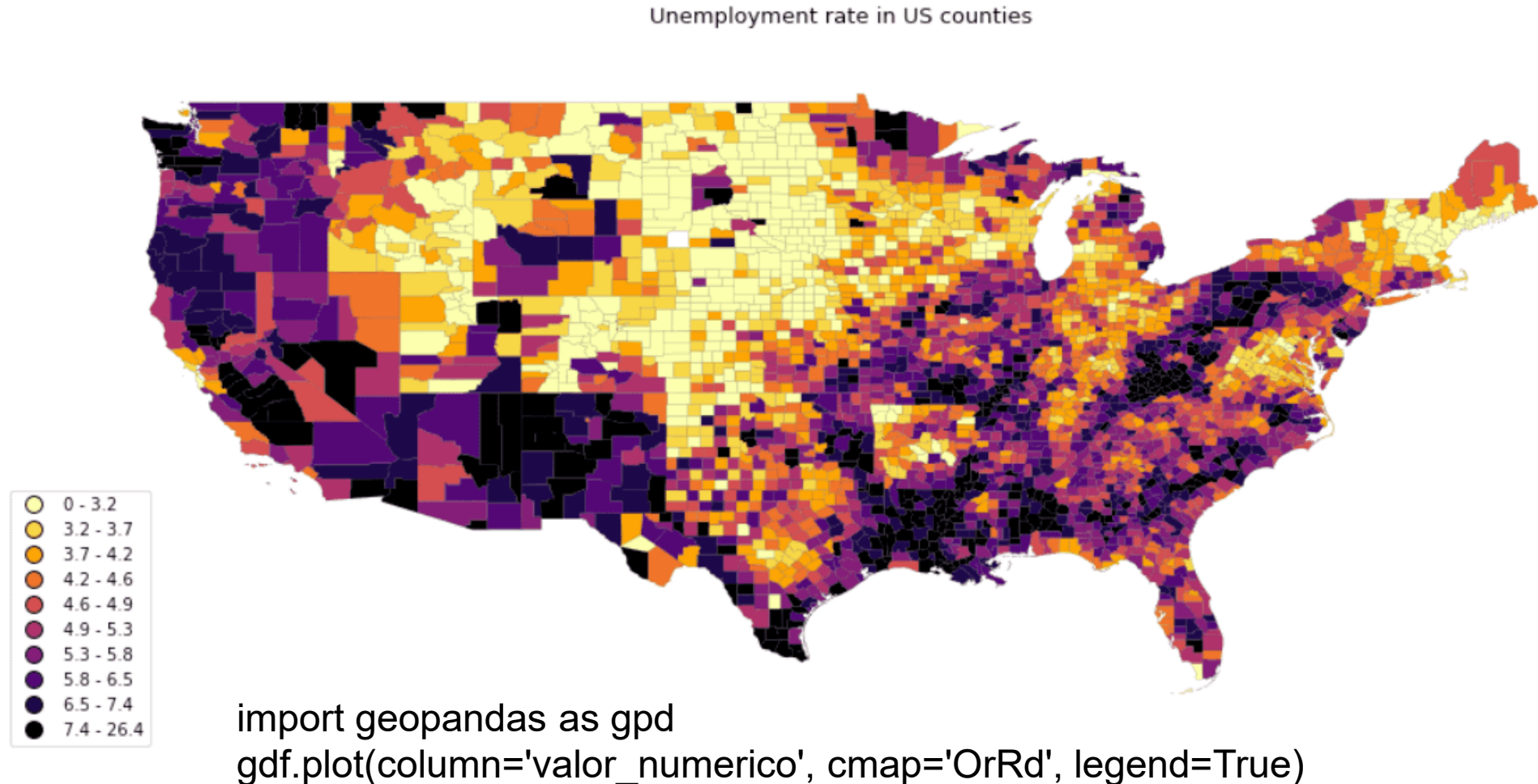
```
import upsetplot
upsetplot.plot(datos_formateados)
```

6. Datos espaciales y geográficos

Visualización de variables mapeadas sobre límites políticos o coordenadas

Mapa coroplético (Choropleth)

Pinta los límites geográficos (GeoJSON) según el valor agregado. Es el más famoso para ver tasas o proporciones por regiones.



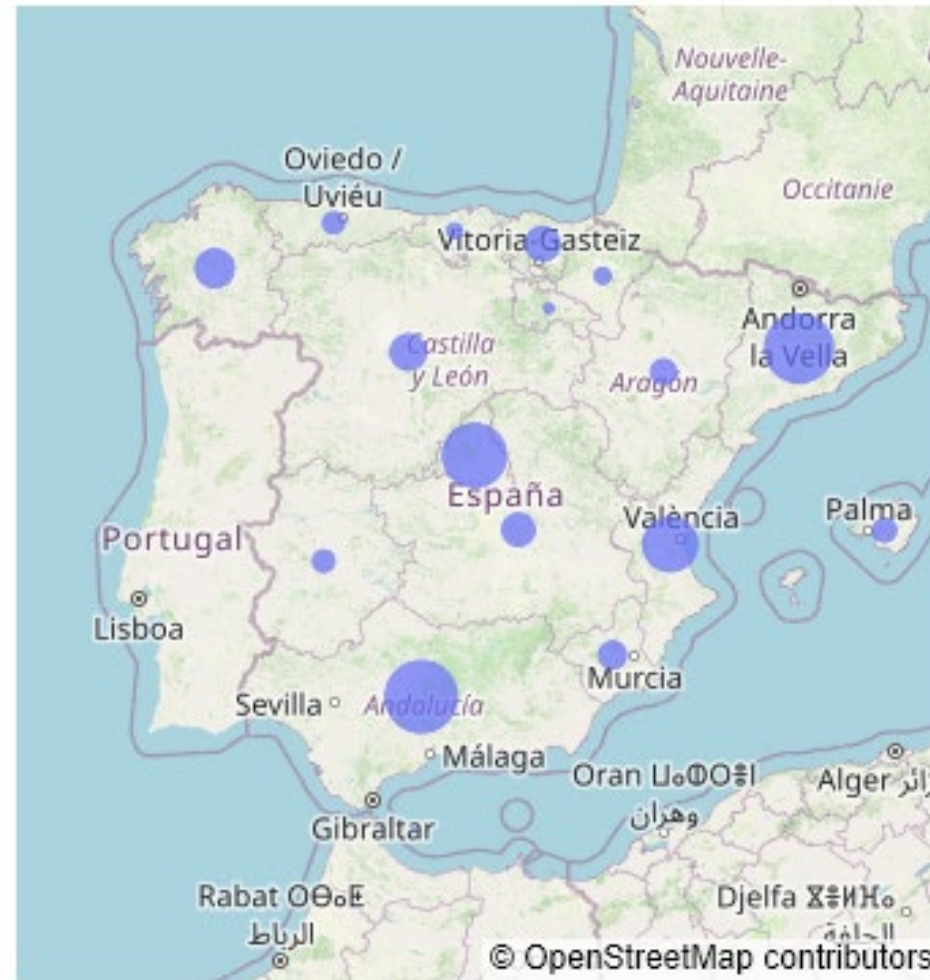
6. Datos espaciales y geográficos

Visualización de variables mapeadas sobre límites políticos o coordenadas

Mapa de burbujas (Bubble map)

Dibuja círculos proporcionales sobre las coordenadas geográficas exactas de cada evento. Excelente para datos puntuales.

Ventaja: Muestra magnitud exacta sobre coordenadas reales sin sobreestimar visualmente regiones despobladas.

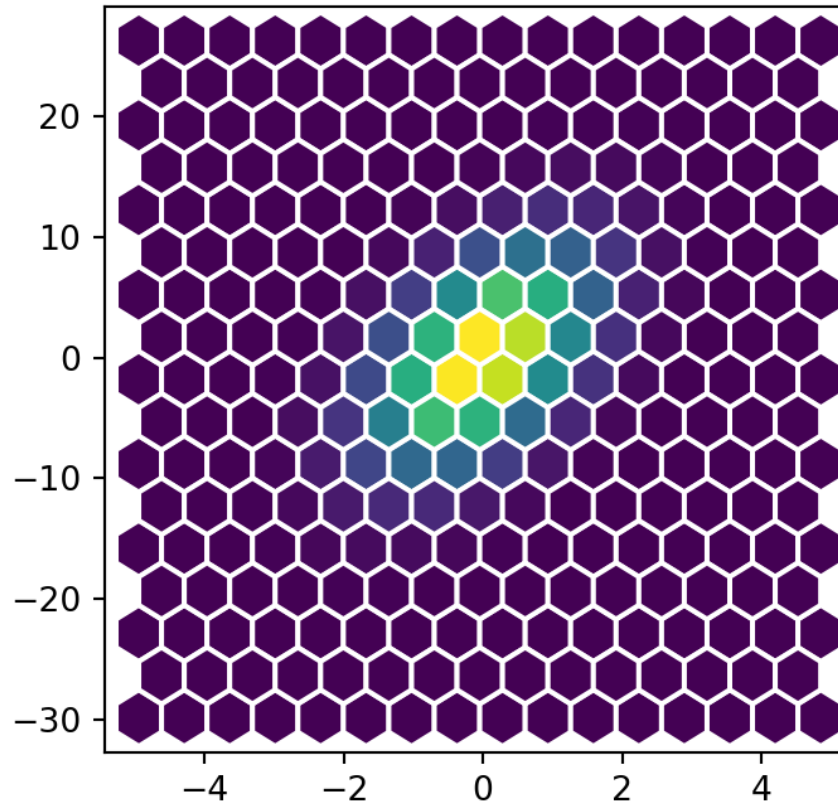


```
import geopandas as gpd
gdf.plot(markersize=
gdf['magnitud'] * 10,
alpha=0.5)
```

6. Datos espaciales y geográficos

Visualización de variables mapeadas sobre límites políticos o coordenadas

Hexbin: Agrupa miles de puntos continuos en celdas hexagonales limpias.



```
import matplotlib.pyplot as plt
plt.hexbin(df['longitud'], df['latitud'], gridsize=50, cmap='inferno')
```

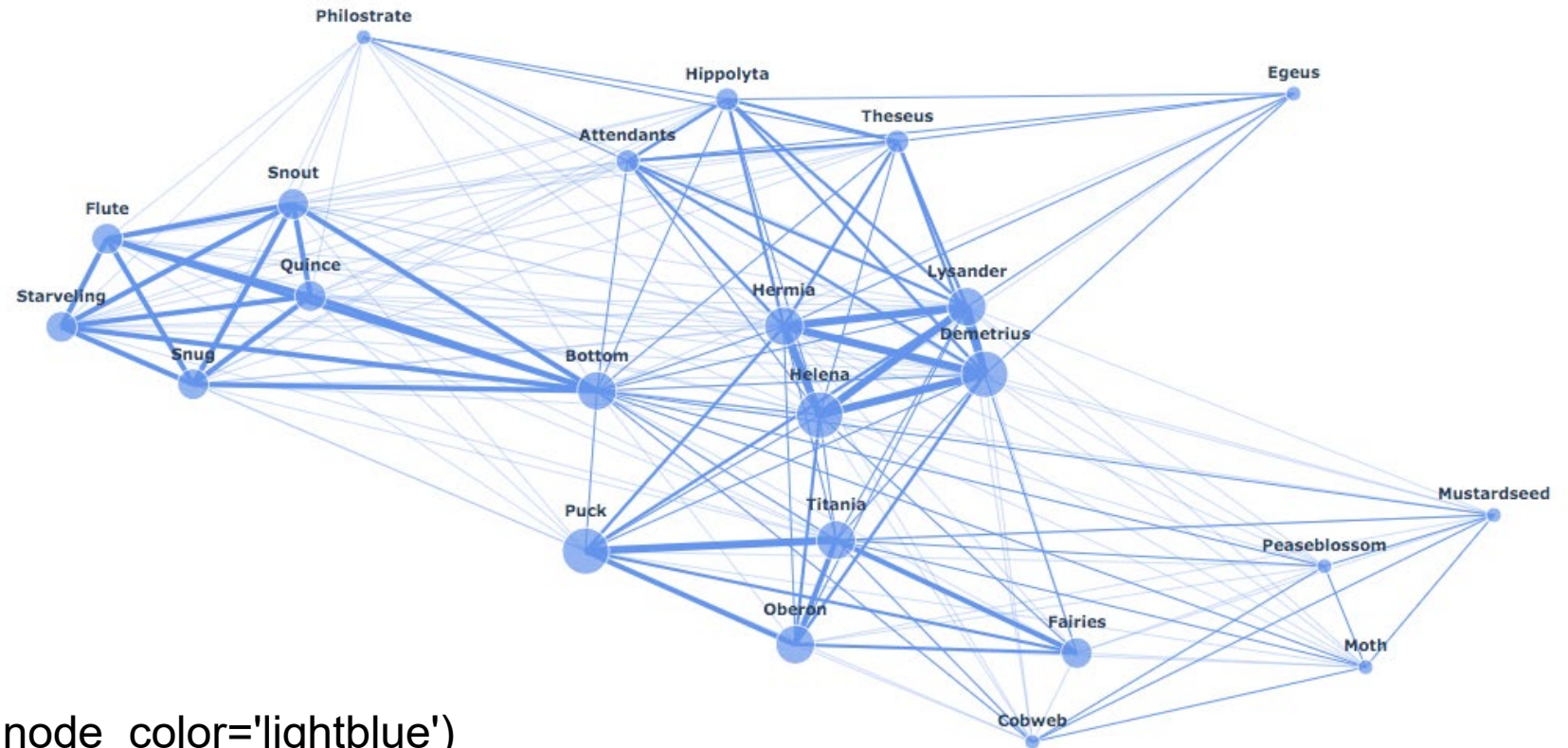
7. Datos relacionales y conexiones

Mapear enlaces complejos, flujos comerciales y estructuras de red

Diagramas de red (Network graphs)

Modelan entidades (nodos o vértices) y relaciones mutuas (enlaces o aristas). Ideales para redes sociales, topología física o interacciones moleculares.

```
import networkx as nx
nx.draw(G, with_labels=True, node_color='lightblue')
```



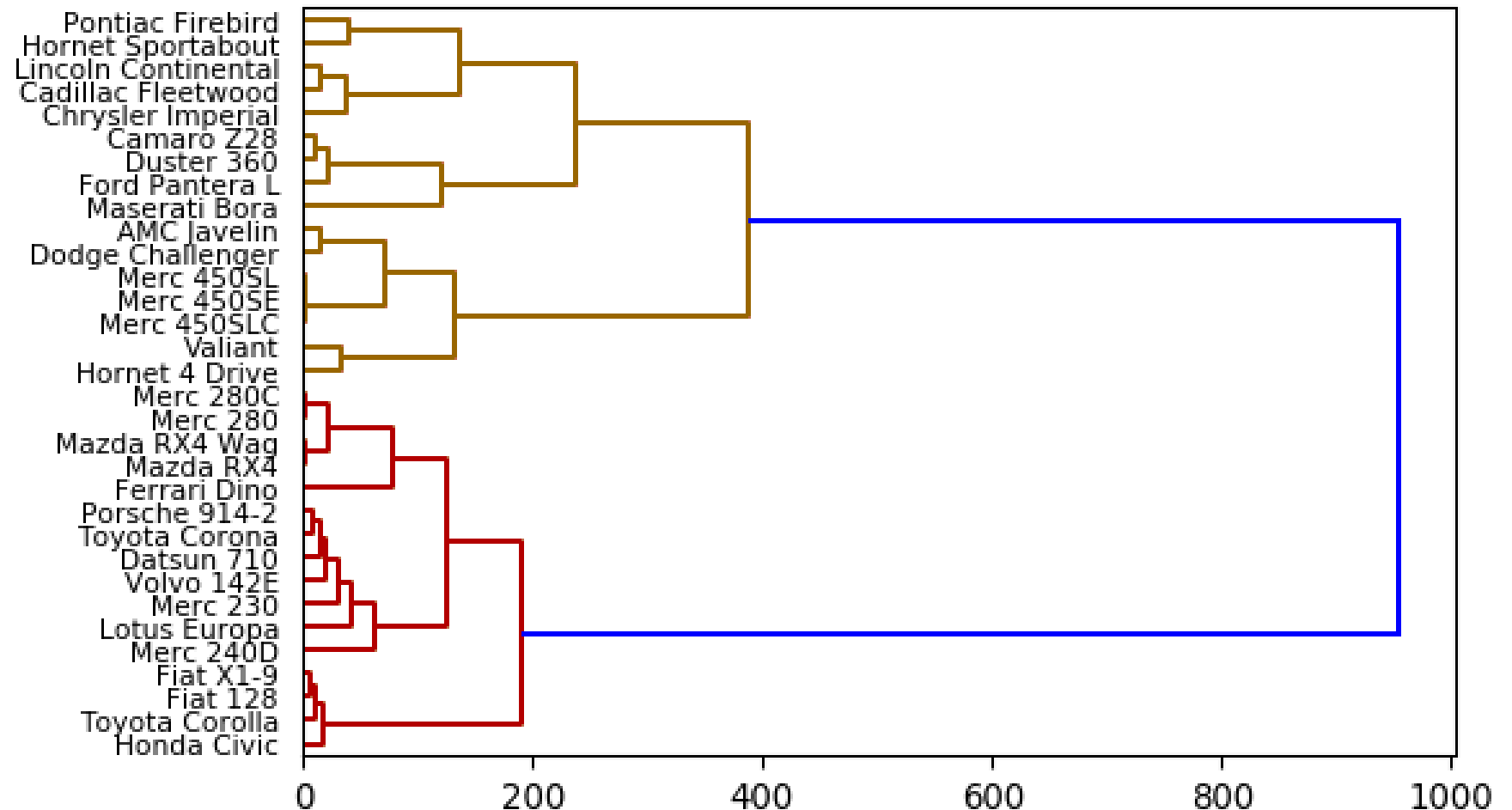
7. Datos relacionales y conexiones

```
from scipy.cluster.hierarchy import dendrogram, linkage
dendrogram(linkage(df, method='ward'))
```

Mapear enlaces complejos, flujos comerciales y estructuras de red

Dendrograma

Diagrama de árbol que mapea jerarquías y similitudes estructurales. Es la salida visual directa para evaluar y definir cortes en algoritmos de Clustering Jerárquico.



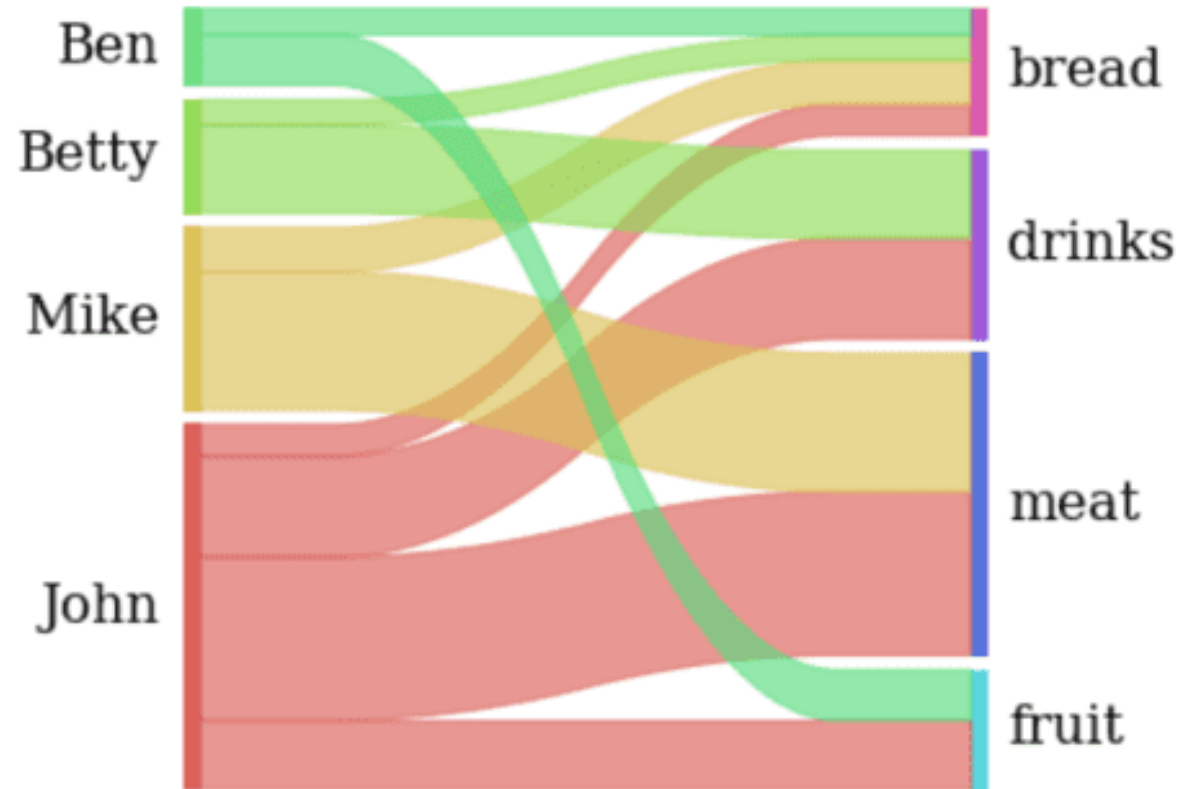
7. Datos relacionales y conexiones

Mapear enlaces complejos, flujos comerciales y estructuras de red

Diagrama de Sankey

Representa la transferencia y flujos continuos desde fuentes iniciales hasta sumideros finales.

Gran usabilidad: Las anchuras de las bandas son directamente proporcionales al volumen del flujo. Ideal para flujos de caja, presupuestos y pérdidas.



```
import plotly.graph_objects as go
go.Figure(go.Sankey(node=dict(label=['Origen', 'Destino 1', 'Destino 2']),
link=dict(source=[0, 0], target=[1, 2], value=[100, 50])))
```

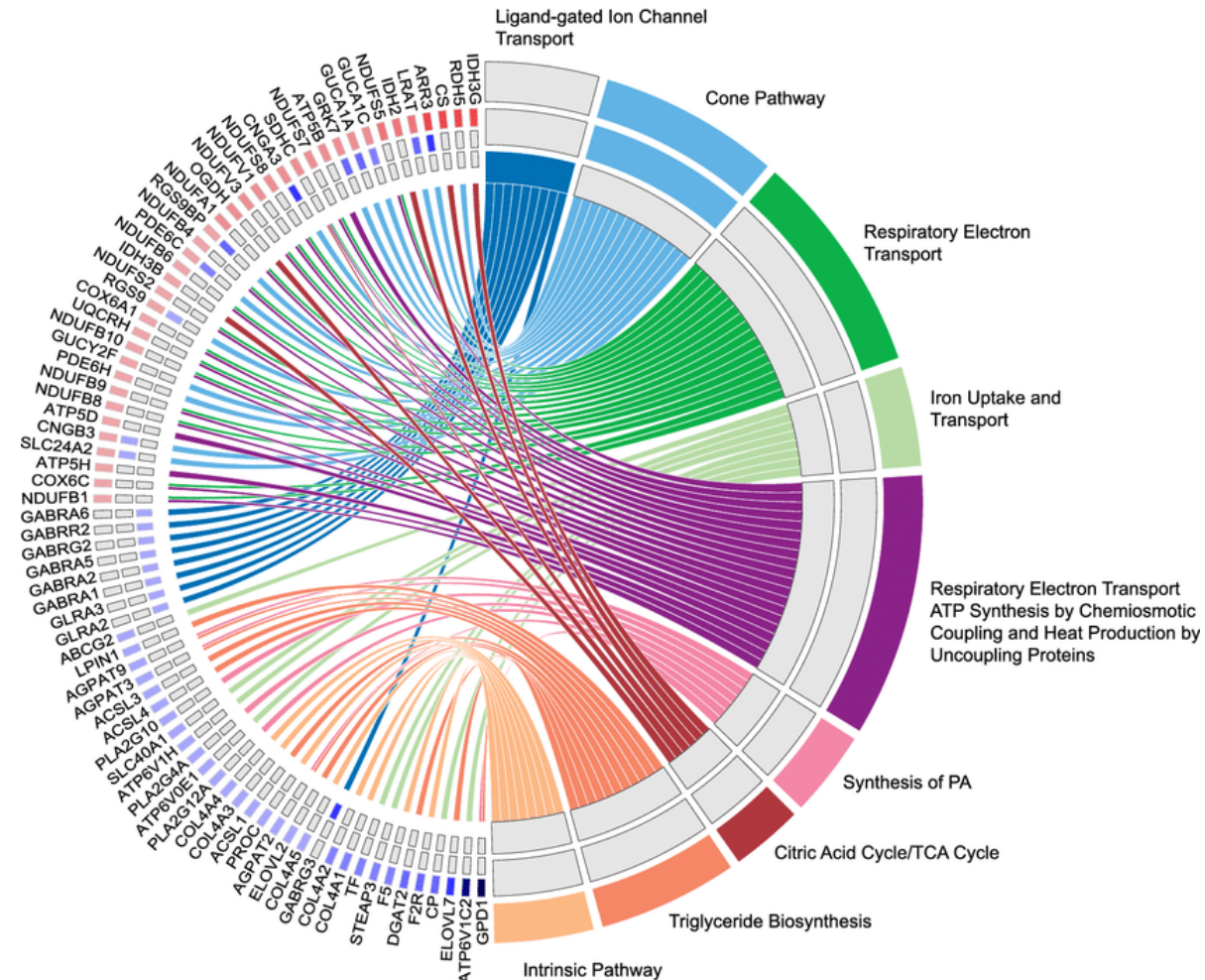
7. Datos relacionales y conexiones

Mapear enlaces complejos, flujos comerciales y estructuras de red

Diagramas de cuerdas (Chord)

Mapear flujos bidireccionales recíprocos entre grupos en un diseño circular.

```
import holoviews as hv
hv.extension('bokeh')
# df_enlaces debe tener columnas:
# ['origen', 'destino', 'valor']
hv.Chord(df_enlaces)
```



7. Datos relacionales y conexiones

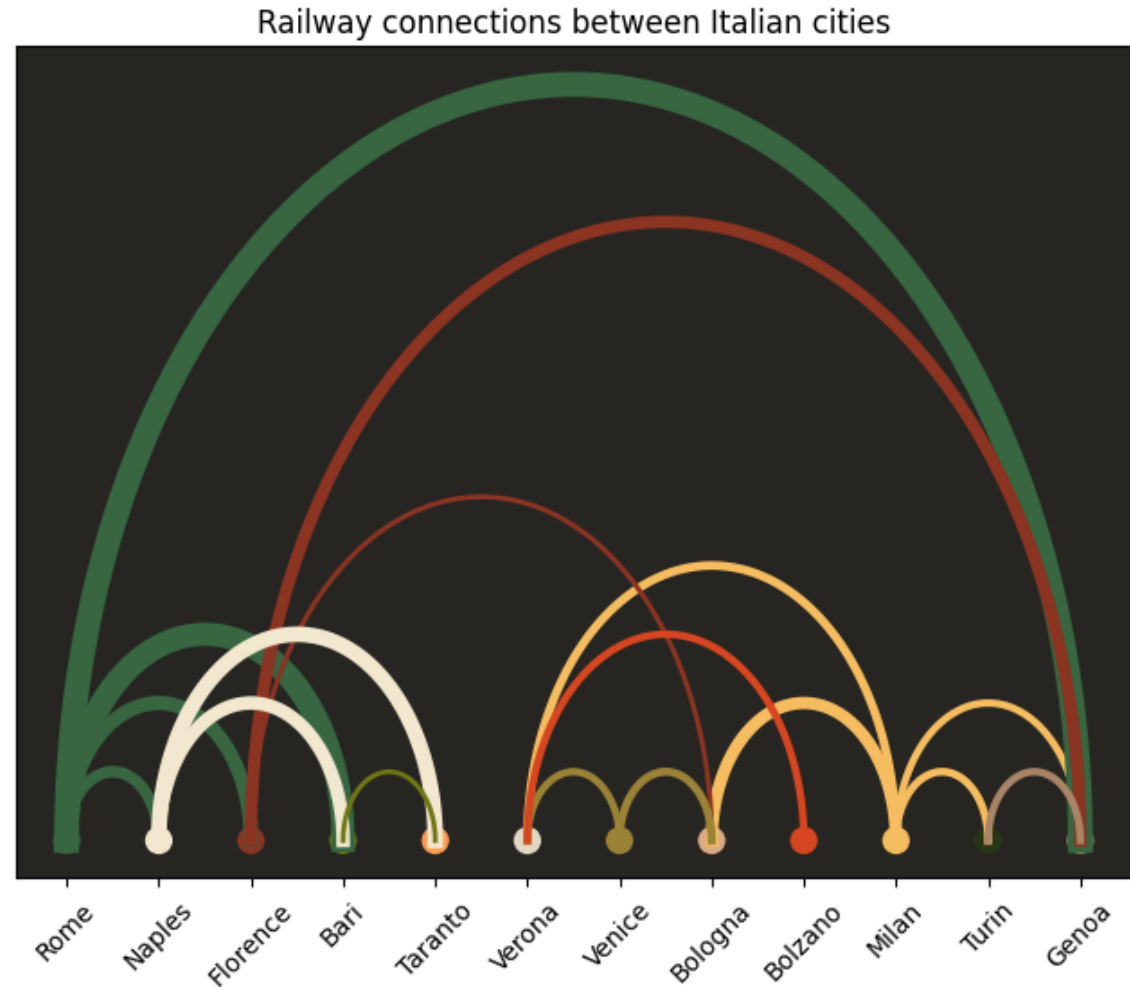
Mapear enlaces complejos, flujos comerciales y estructuras de red

Arc diagram

Una simplificación lineal de red.

Enfoque: Úsalos en aplicaciones de inteligencia de red o balances de importación/exportación de comercio.

```
import networkx as nx
from nxviz import ArcPlot
# grafo_nx es un objeto de red de NetworkX
ArcPlot(grafo_nx, node_color='grupo',
edge_width='peso').draw()
```



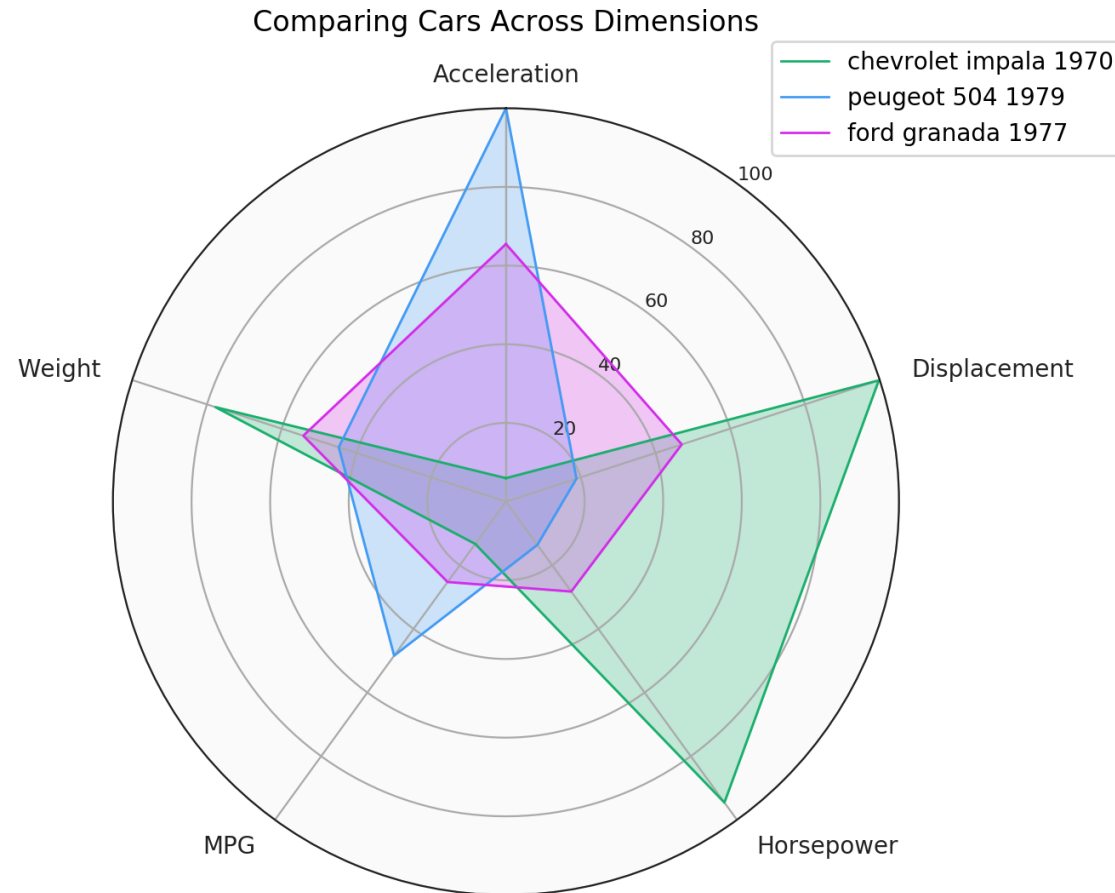
Antipatrones: Gráficos a evitar

Visualizaciones problemáticas con baja precisión de lectura o distorsión cognitiva

Gráficos de radar (Spider charts)

La forma irregular generada depende por completo del orden de las variables en los ejes. Además, el ojo humano sobreestima el área sombreada de las esquinas.

Evitar siempre en comparaciones cuantitativas directas. Reemplazar por coordenadas paralelas.



```
import plotly.express as px
px.line_polar(df, r='valores', theta='categorias', line_close=True)
```

Antipatrones: Gráficos a evitar

Visualizaciones problemáticas con baja precisión de lectura o distorsión cognitiva

Nubes de palabras (Wordclouds)

El área ocupada depende del tamaño del string y no solo de su peso real de frecuencia. Dificulta comparar categorías numéricamente.

Evitar como gráfico analítico riguroso. Sustituir por un barplot horizontal ordenado.



```
from wordcloud import WordCloud
nube = WordCloud(width=800, height=400).generate(texto_string)
```

Antipatrones: Gráficos a evitar

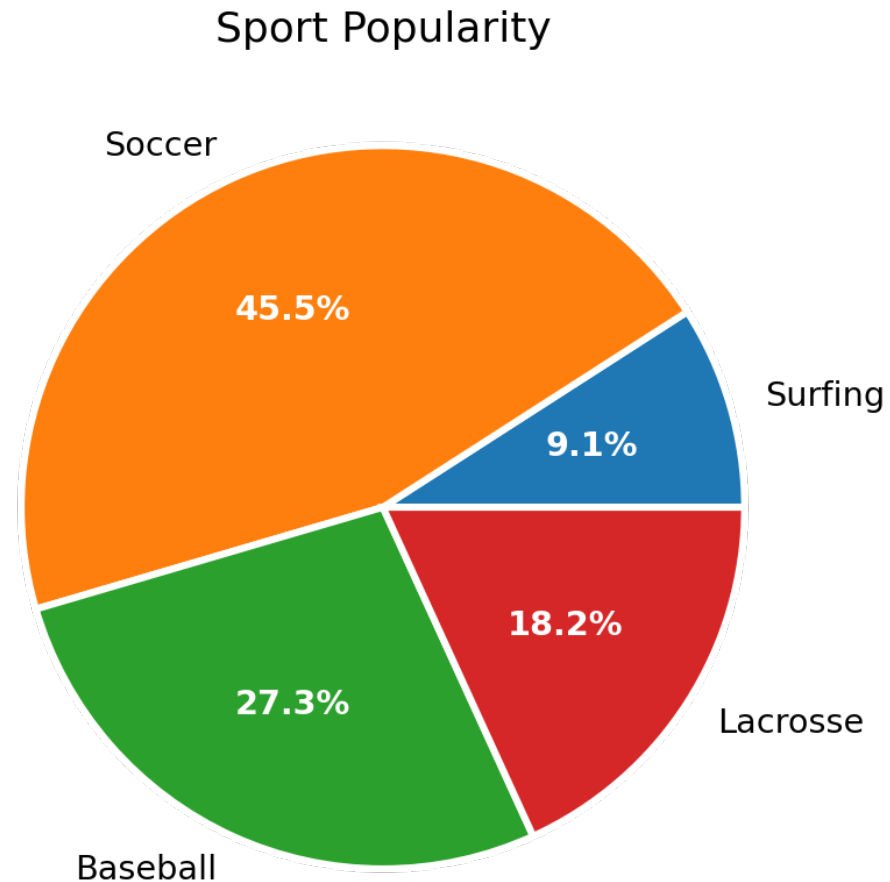
Visualizaciones problemáticas con baja precisión de lectura o distorsión cognitiva

Gráficos circulares (Pie charts)

El cerebro humano no es preciso decodificando ángulos ni áreas circulares. Tiende a subestimar u ocultar diferencias muy finas entre categorías.

Prohibido con > 5 categorías. Solo usar si se necesita representar partes de un todo de forma rápida.

```
import matplotlib.pyplot as plt
plt.pie(df['valor'], labels=df['categoria'], autopct='%1.1f%%')
```



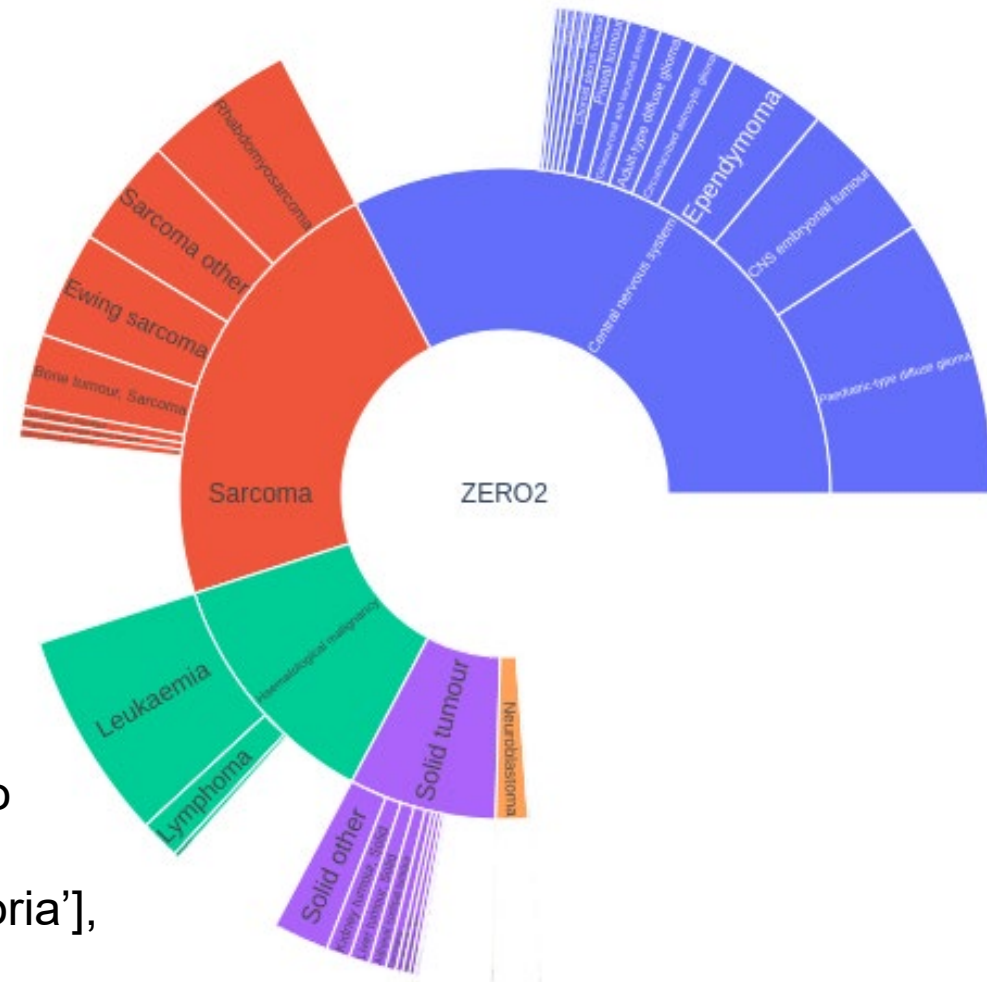
Antipatrones: Gráficos a evitar

Visualizaciones problemáticas con baja precisión de lectura o distorsión cognitiva

Sunburst (Rayo de sol)

Sectores anidados en múltiples niveles concéntricos. Distorsiona gravemente la percepción del área; es preferible sustituirlo siempre por un Treemap rectangular.

```
import plotly.express as px
# 'path' define el orden de los anillos desde el centro
# hacia afuera
px.sunburst(df, path=['Categoria_Padre', 'Subcategoria'],
values='magnitud')
```



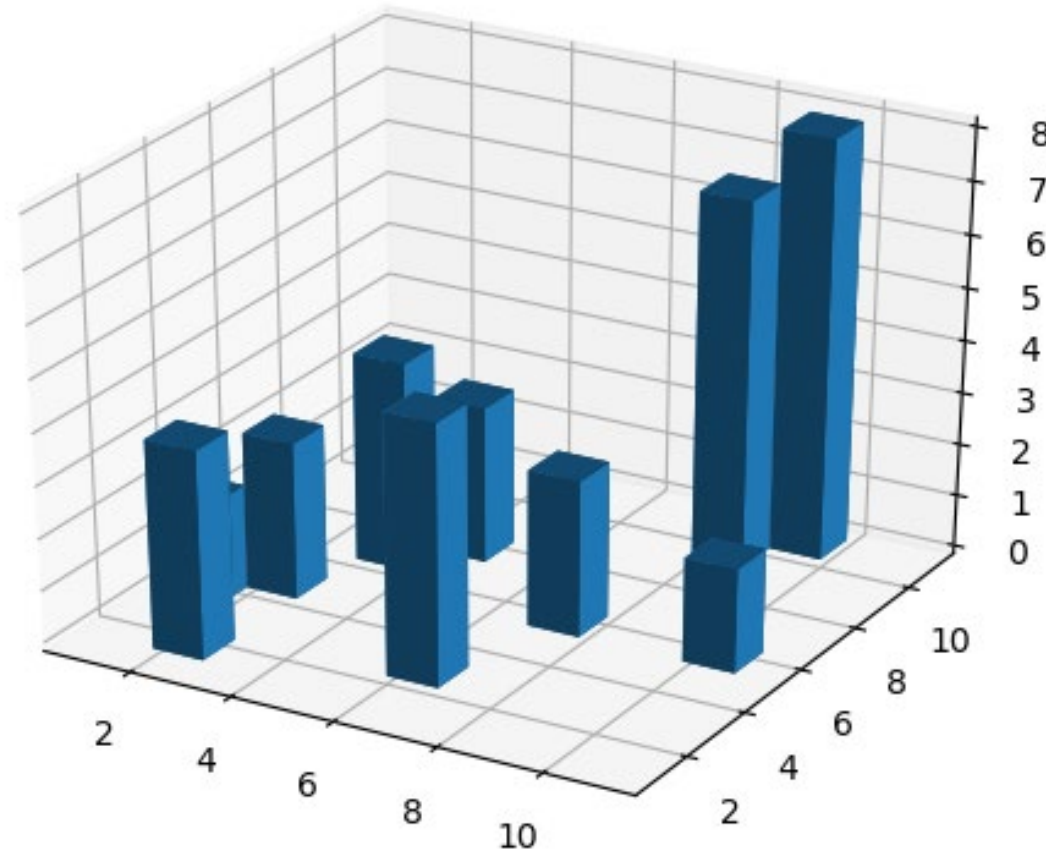
Antipatrones: Gráficos a evitar

Visualizaciones problemáticas con baja precisión de lectura o distorsión cognitiva

Perspectiva 3D (Plots en 3D)

Los gráficos 3D (barras o sectores 3D) distorsionan la magnitud espacial para crear una ilusión óptica inútil. La perspectiva altera los valores reales.

Un gráfico bidimensional limpio es superior.



```
import matplotlib.pyplot as plt
fig = plt.figure()
ax = fig.add_subplot(projection='3d') # Activa el eje 3D
ax.bar3d(x, y, z_base, dx, dy, z_altura) # Para barras 3D (x, y, base_z, ancho_x, prof_y, alto_z)
```

Guía de selección: El gráfico correcto

Matriz sintética de decisión según el objetivo y el tipo de variable

Objetivo analítico	Tipo de variable	Gráfico recomendado	Directriz clave
Evolución temporal	Numérica + Tiempo	Gráfico de líneas (Lineplot)	Eje X ordenado estrictamente en secuencia temporal.
Comparación grupos	Categórica + Magnitud	Barras horizontales (o Lollipop)	Usa horizontal si los nombres son largos. Etiqueta directo.
Distribución frecuencia	Numérica + Grupos	Boxplot, Violín, o Crestas	Boxplot para ver outliers, violín para ver la forma de la distribución y crestas para compararla en muchas categorías.
Correlación / Relación	Numérica + Numérica	Dispersión (Scatterplot) con regresión	Usa transparencias o Hexbins si hay solapamiento masivo.
Flujos y procesos	Flujos cuantitativos	Diagrama de Sankey o Grafos	Ancho de banda proporcional al flujo. Nodos distribuidos.

Cheatsheet de gráficos estáticos
